

这份 Python 脚本实现了一个基于混合整数线性规划 (MILP) 与动态权重迭代的地面探测站 (DSN) 多航天器检测任务调度系统。

其核心目标是在满足天线资源容量、天线级冲突以及任务级互斥等硬性约束的前提下，最大化高优先级任务的满意度，并辅以动态反馈机制来逐步消除欠调度任务。

以下是代码中所有算法构造的详细拆解：

算法 1：基于 XOR 拆分的任务预处理与建模 (solve)

代码采用 MILP (Mixed-Integer Linear Programming) 来处理核心的资源分配问题。其对应的数学模型构建（即代码注释中提及的 Equation 6a-6m）如下：

1. 决策变量 (Decision Variables)

对于每个活动 (Activity)，引入一个二进制决策变量 x_a （代码中的 `x[a_id]`）：

$x_a \in \{0, 1\}$

- 若 $x_a = 1$ ，代表该活动被选中并加入初步调度池。
- 若 $x_a = 0$ ，则放弃该活动。

2. 任务的 XOR 拆分逻辑 (XOR Splitting)

为了灵活应对长时间的窗口占用，算法支持任务拆分（代码中对应 Algorithm 1 逻辑）：

- 如果一个活动允许拆分 (`act['can_split'] == True`)，算法会派生出两个子活动 a' (`a_prime`) 和 a'' (`a_dprime`)。
- 成对调度约束 (Constraint 6k, 6l)：子活动必须同时被调度或同时放弃：
 $x_{a'} = x_{a''}$
- 互斥约束 (Constraint 6m)：原始活动与拆分后的子活动是互斥的，不能同时存在：
 $x_a + x_{a'} \leq 1$

3. 物理资源总量约束 (Capacity Constraint 6i)

为了防止求解器无节制地塞入任务，模型建立了一个总资源容量上限。所有被选中的活动（或半时长的拆分活动）的持续时间之和，不能超过总天线资源池的上限：

$\sum (Dur_a \cdot x_a) \leq \text{总天线数} (40) \times \text{总时长} (168h) \times \text{最大利用率} (70\%)$

4. 优先级加权的复合目标函数 (Objective Function 6a)

模型的目标是最大化全局调度效益。为了让高优先级和高权重的任务被优先选择，目标函数中的每个变量系数都经过了复合加权：

$\max \sum_a \left(W_{\text{iter}} \times W_{\text{priority}} \times W_{\text{split}} \times x_a \right)$

- W_{iter} ：算法 2 迭代中赋予该任务的动态更新权重（初始为 1.0）。
- W_{priority} ：活动的优先级权重。代码采用了平方映射 priority^2 （将 1-5 级映射为 1, 4, 9, 16, 25）。这种非线性放大能确保高优先级任务在遭遇资源冲突时，获得绝对的数学优势。

- `$W_{\text{split}}`：如果是拆分任务，权重减半（0.5），鼓励求解器优先选择完整、连续的非拆分任务。

算法 2：基于滑动阈值的动态权重迭代算法 (`run_dynamic_optimization`)

单纯的 MILP 求解往往会导致“富者更富，贫者更贫”——高优先级任务吃满资源，部分低优先级任务满意度直接挂零。为了改善全局公平性，代码实现了一套**动态反馈迭代机制**。

```
[初始化] 任务权重  $w = 1.0$ ，初始满意度阈值  $\eta = 15\%$ 
|
▼
循环迭代  $k = 1$  到  $k_{\max}$ :
|
|— 1. 调用 MILP 求解器，计算当前权重下的全局最优解
|— 2. 计算每个航天器任务的满意度 = 成功调度时长 / 总申请时长
|
|— 3. 检查是否有任务的满意度 < 阈值  $\eta$  ?
|   |— YES (存在欠调度任务):
|   |   对这些任务进行惩罚性大加码:  $w = w * 2.0$ 
|   |— NO (所有任务均达标):
|   |   说明当前状态良好，提高挑战，抬高满意度门槛:  $\eta = \eta + 5\%$ 
|
▼
[输出] 结束循环，交付最终优化结果
```

核心设计机理：

1. **惩罚性权重翻倍** (`weights[m_id] *= 2.0`)：在下次 MILP 求解时，这些“挂科”任务的系数在目标函数中暴涨，迫使求解器挪用其他资源来优先满足它。
2. **动态门槛跃升** (`eta += eta_plus`)：当资源较为宽松、全员达标时，算法会自动收紧约束（门槛加 5%），榨干探测站的最后一点闲置吞吐量。

算法 3：基于优先级贪心的两阶段冲突消解算法 (`_resolve_conflicts`)

由于核心 MILP 变量 `x[a_id]` 仅决定了**活动“选不选”**以及**天线资源“够不够”**（采用大粒度的总量约束），并没有在求解器内做精准到具体分钟的排程。

为了避免在特定时间段内出现天线撞车，代码在 MILP 求解完成后，引入了第二阶段的**确定性冲突消解算法**：

步骤一：多维复合排序（贪心准备）

算法将 MILP 筛选出的候选方案（包含了活动在所有可选观测窗口 `view_periods` 里的展开形态）收集起来，按照以下规则进行**降序排列**：

1. **主关键字**：航天器任务的优先级（从高到低）。
2. **次关键字**：活动的开始时间（从早到晚）。

步骤二：时间线碰撞检查与锁入

维护两张动态时间表：`antenna_timeline`（天线占用表）和 `mission_timeline`（航天器自身占用表）。遍历排序后的活动，对每一个活动执行**硬约束检查**：

1. **天线级时空冲突 (Constraint 6h)：**

计算该活动包含建链准备 (Setup) 和拆除归还 (Teardown) 后的完整时间闭包 $[Start - Setup, End + Teardown]$ 。检查目标天线在此区间内是否已被更高优先级的活动占用。

2. **航天器单射频互斥 (Constraint 6j)：**

检查该航天器本体是否在同一时间段内被指派给了另一台天线（单航天器不能分身同时与两台天线通信）。

- **判定结果：**若两项检查均无冲突，则将该时间段写入两张时间表，并锁定该活动；若有冲突，由于高优先级任务已经排在前面被处理完了，此时**直接舍弃当前的低优先级活动**。

辅助构件

1. 时间覆盖加权评估 (`_get_activity_priority`)

由于一个航天器的优先级在 168 小时内可能会动态变化（例如遭遇飞掠关键事件时突然升级），该函数计算一个活动所覆盖的所有可见窗口（`view_periods`），并与动态优先级时间表（`priority_schedule`）做交集计算，最终输出该活动在整个周期内的**时间加权平均优先级**，作为 MILP 目标函数的系数底数。

2. 甘特图可视化引擎 (`visualize_and_save`)

利用 `matplotlib` 库将最终的数据表翻译成直观的探测站排班甘特图：

- **灰色区块：**代表天线对准与锁定的 Setup / Teardown 闲置开销。
- **彩色区块：**代表各航天器 (Mission) 实际在轨跟踪与数据下行的时段。
- **纵轴/横轴：**纵轴对应不同天线标识，横轴对应周一 00:00 展开的一周 168 小时全景线。